EST-744

Mecanismos de Atención

Juan Zamora O.

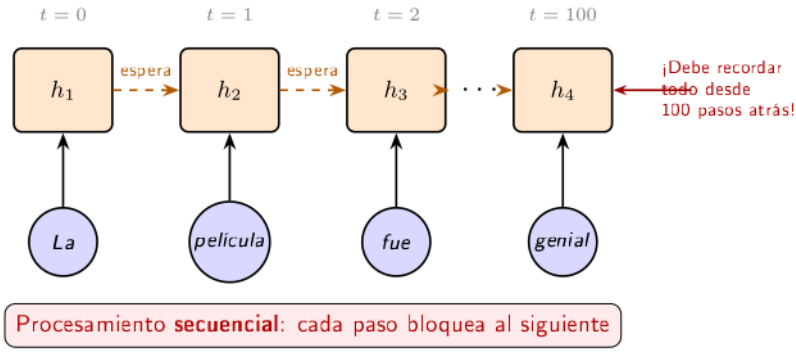
Deep Learning





El Problema: Cuello de Botella Secuencial

Las RNN procesan una palabra a la vez, creando un cuello de botella:



Problemas:

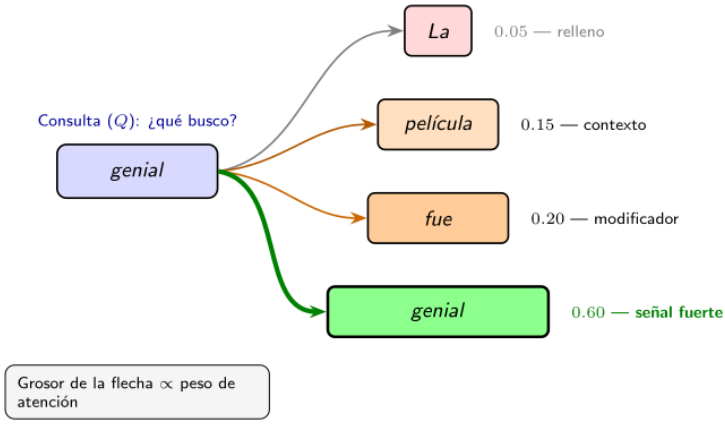
- No se puede paralelizar (hay que procesar en secuencia)
- El contexto lejano se desvanece (gradientes desvanecientes)
- Entrenamiento lento en secuencias largas

¿Qué pasaría si cada palabra pudiera "mirar" a todas las demás al mismo tiempo?

Vaswani et al. (2017), "Attention Is All You Need", NeurIPS. El artículo que introdujo los Transformers — disponible en `arxiv.org/abs/1706.03762`

Selección Suave

En lugar de leer de izquierda a derecha, la **atención** permite enfocarse en las palabras relevantes:



Enfoque selectivo del mecanismo de Atención:

Componente	Pregunta que responde
Consulta (***Query***) Q	"¿Qué estoy tratando de entender?"
Clave (***Key***) K	"¿Qué es cada palabra?"
Valor (***Value***) V	"¿Qué información contiene cada palabra?"
Salida	Promedio ponderado de las palabras relevantes

Bahdanau, Cho & Bengio (2015), "Neural Machine Translation by Jointly Learning to Align and Translate", ICLR. El artículo original de atención — `arxiv.org/abs/1409.0473`

Notación y su significado

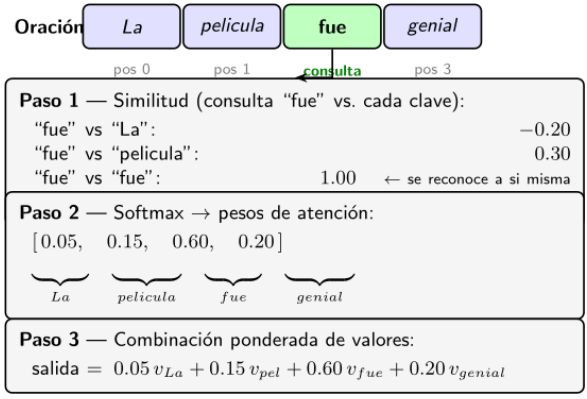
$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Paso a paso:

1. **Consulta** Q: proyección lineal de la posición actual — "¿qué busco?"
2. **Producto** QK^T: similitud entre la consulta y cada clave — "¿cuánto encaja cada palabra?"
3. **Escala** $/\sqrt{d_k}$: evita que los productos internos crezcan demasiado y saturen el softmax
4. **Softmax**: convierte similitudes en pesos de atención (entre 0 y 1, suman 1)
5. **Valores** V: combinación ponderada de los vectores de palabras

Vaswani et al. (2017), sección 3.2. Para una derivación paso a paso: Alammar (2018), "The Illustrated Transformer" — https://jalammar.github.io/illustrated-transformer/

Ejemplo de Atención sobre “fue”

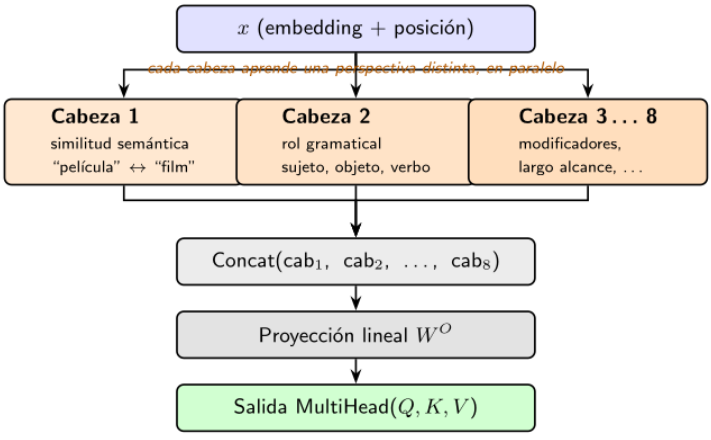


La representación final de “fue” incorpora información de toda la oración, especialmente de “genial” — capturando la relación semántica entre verbo y adjetivo.

Múltiples Perspectivas Simultáneas

Una sola cabeza de atención no captura todas las relaciones posibles.

Solución: usar varias cabezas (***heads***), cada una aprendiendo patrones distintos:



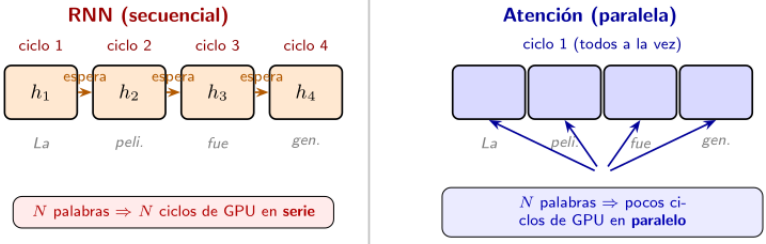
Las salidas de todas las cabezas se **concatenan** y se proyectan linealmente:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{cabeza}_1, \ldots, \text{cabeza}_h)\, W^O$$

¿Por qué múltiples cabezas? - Captura distintas dimensiones del significado en paralelo - Robustez: si una cabeza falla, las demás compensan

Vaswani et al. (2017), sección 3.3. En el paper original se usan $h = 8$ cabezas con $d_k = d_v = 64$.

RNN vs. Atención



Ganancia de velocidad: **10–50**× en secuencias largas

Métrica	RNN/LSTM	Atención
Procesamiento	Secuencial	**Paralelo**
Ganancia en velocidad	1× (base)	**10–50×** en secuencias largas
Uso de GPU	Bajo (un paso a la vez)	**Alto** (todos los pares)

¿Cómo Sabe la Atención el Orden?

La atención trata todas las palabras por igual: no tiene noción de posición.

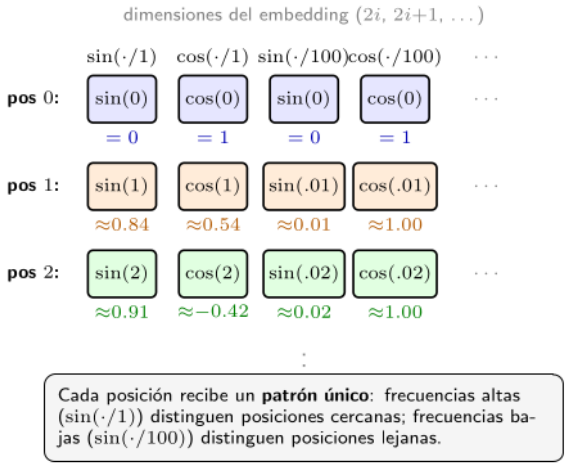
Problema: “El perro mordió al hombre” ≠ “El hombre mordió al perro” producirían la misma representación sin información posicional.

Solución: sumar una **codificación posicional** (Positional Encoding) al embedding de cada palabra:

$$PE(\text{pos},\, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

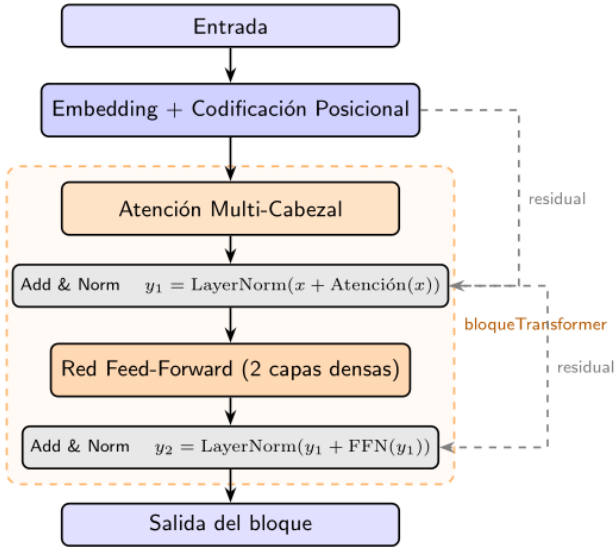
$$PE(\text{pos},\, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

Cada posición recibe un patrón sinusoidal único que varía a distintas frecuencias:



El uso de sinusoides permite al modelo generalizar a secuencias más largas que las vistas en entrenamiento. Ver Vaswani et al. (2017), sección 3.5.

Las Piezas Ensambladas



Conexión residual y normalización (***Add & Norm***):

$$\text{Salida} = \text{LayerNorm}(x + \text{Atención}(x))$$

¿Por qué residual? ***Add*** toma la entrada original de una capa y la suma directamente a la salida procesada de esa misma capa. Ayuda a preservar la información original y evita que se pierda a medida que el modelo se vuelve más profundo.

Luego se aplica una técnica de normalización al resultado combinado. Esto estabiliza el flujo de gradientes, evitando que sean demasiado grandes o pequeños.

He et al. (2016), "Deep Residual Learning for Image Recognition" — origen de las conexiones residuales, aplicadas luego a transformers.

Usar la Capa Incorporada

TensorFlow 2.x incluye `MultiHeadAttention` lista para usar:

```
capa_atencion = keras.layers.MultiHeadAttention(
    num_heads=8,        # numero de cabezas independientes
    key_dim=64          # dimension por cabeza (total = key_dim x num_heads)
)

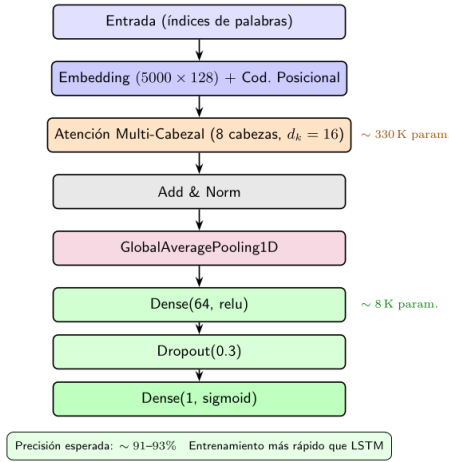
# Auto-atencion sobre si misma
salida = capa_atencion(query=x, value=x, key=x)
```

Parámetros clave:

Parámetro	Significado	Valor típico
`num_heads`	Cabezas independientes	4–16
`key_dim`	Dimensión por cabeza	32–128
`dropout`	Regularización en los pesos	0.1–0.3

Tarea y Arquitectura

Actividad: clasificar reseñas de IMDb como positivas/negativas usando atención con la siguiente arquitectura.

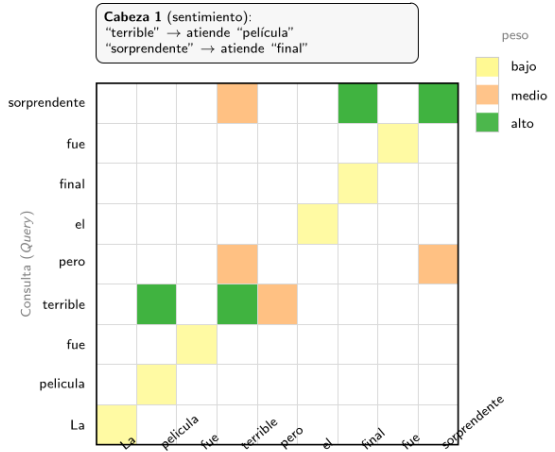


Ventajas de la atención

- Precisión comparable a LSTM
- Mucho más rápido (paralelizable)
- Mejor escalabilidad a secuencias largas
- **Interpretable**: los pesos de atención muestran en qué se enfocó el modelo

¿Qué Está Mirando el Modelo?

Los pesos de atención permiten **interpretar** las decisiones del modelo:



Relevancia de este mecanismo

Interpretabilidad es una ventaja concreta sobre LSTM, donde los estados ocultos son difíciles de leer directamente.

La atención es el **mecanismo central** de BERT, GPT y todos los modelos modernos de NLP. Comprender su funcionamiento es imprescindible para la Clase 5.

Clark et al. (2019), “What Does BERT Look at?”, ACL. Análisis detallado de qué aprenden las cabezas de atención en modelos pre-entrenados.

Diagnóstico

Problema	Causa	Solución
Toda la atención a una sola palabra	Entrenamiento insuficiente	Entrenar más épocas; aumentar tasa de aprendizaje
Atención uniforme (sin foco)	Modelo muy simple	Agregar más cabezas o capas más profundas
Error de memoria	Muchas cabezas × dimensiones altas	Reducir `key_dim` o `num_heads`
Entrenamiento lento	Implementación ineficiente	Usar capas integradas de Keras, no implementación manual
Sobreajuste severo	Pocos datos o modelo muy grande	Agregar `dropout` en atención y capas densas

Cierre

Los mecanismos de atención permiten que cada palabra “vea” a todas las demás de forma instantánea y paralela

Esta combinación de expresividad y eficiencia computacional es la razón por la que los Transformers dominan las aplicaciones modernas de NLP

Para Profundizar

- **Vaswani et al. (2017).** “Attention Is All You Need.” ***NeurIPS***. `arxiv.org/abs/1706.03762` — El artículo fundacional de los Transformers; lectura obligatoria.
- **Bahdanau, Cho & Bengio (2015).** “Neural Machine Translation by Jointly Learning to Align and Translate.” ***ICLR***. `arxiv.org/abs/1409.0473` — Origen del mecanismo de atención para secuencias.
- **Alammar, J. (2018).** “The Illustrated Transformer.” `jalammar.github.io/illustrated-transformer` — La explicación visual más recomendada; sin necesidad de código previo.
- **Goodfellow, Bengio & Courville (2016).** ***Deep Learning***, Cap. 12. MIT Press. `deeplearningbook.org` — Marco teórico de modelos de secuencias.
- **Clark et al. (2019).** “What Does BERT Look at?” ***ACL Workshop***. `arxiv.org/abs/1906.04341` — Análisis empírico de qué aprenden las cabezas de atención.

Tareas Prácticas

1. **Visualizar atención**: obtener pesos de atención para 10 reseñas. ¿Los patrones tienen sentido lingüístico?
2. **Variar `num_heads`**: probar 2, 4, 8 y 16 cabezas. ¿Más siempre es mejor?
3. **Comparar con LSTM**: entrenar LSTM con los mismos hiperparámetros. ¿Cuánto gana la atención en velocidad?
4. **Ablación posicional**: entrenar sin codificación posicional. ¿Cuánto impacta en la precisión?
5. **Interpretabilidad**: para reseñas mal clasificadas, ¿qué atendió el modelo? ¿Explica el error?
6. **Escala**: entrenar con secuencias de largo 100, 500 y 1000. Comparar tiempo de época: Atención vs. LSTM.